Learning the football project

The thesis, the code and the viva

This guide explains **Locating a Missing Football from Player Information**, by Maciej Lopacinski, ATU Galway. It starts with the problem, follows the information through the system, explains the evidence and connects each part to the uploaded Python code.

It assumes slight technical knowledge and no previous knowledge of this project. Examples marked **illustration** explain a calculation; they are not extra experimental results. The published numbers come from the submitted thesis, not from a new run.

The sources checked were the complete 45-page **Thesis.pdf uploaded to GitHub**, the complete supplied 31-page learning guide and all 37 current text/code/configuration files in the repository at commit **2c57c02fda176855126c4f883fb527cbc2151481**. The local submitted PDF was checked against GitHub's file identity and matches exactly. The thesis itself has 39 numbered pages after the opening pages.

Read in order the first time. Later, use the contents and PDF bookmarks to return to an explanation. The final sections provide viva questions, a glossary and a short revision page. They are prompts for understanding rather than a script to memorise.

One distinction matters throughout: the uploaded repository is a simplified implementation of the player-based method. Its detection and optional head preparation differ from those used for the published results. Section 22 explains this openly. No code, thesis results or source thesis files were changed to make this guide.

Contents

1. Start with the problem 3
2. The original idea and the pivot 4
3. The data and the split 5
4. Removing the ball without giving away the label 6
5. What YOLO supplies, and why the grass check exists 7
6. What the body points actually have to do with the ball 8
7. How the system learns which player to choose 10
8. How the selected player becomes a ball coordinate 11
9. Raw prediction, blending and the central rectangle 12
10. Why there are seven methods 13
11. What Gaze-LLE is doing in this study 14
12. Understanding error and the close threshold 15
13. Why matches are weighted and resampled 16
14. The original 61-image result 17
15. The 813-image result and the feature evidence 18
16. Why some images are much harder 19
17. Answer-assisted selection, without the confusing name 20
18. What did the two offset models earn? 21
19. The extra edit and replay checks 22
20. The 49-image and earlier heatmap checks 23
21. What the literature contributes to the argument 24
22. The uploaded code and the submitted thesis are not identical 26
23. What every current code file is for 27
24. Follow one image and one player through the code 29
25. Running the current repository and reading its outputs 30
26. Settings and dependencies without mystery 32
27. The contribution, limits and worthwhile next steps 34
28. Viva questions: explain the project first 35
29. Viva questions: data, learning and implementation 36
30. Viva questions: results and the difficult comparisons 38
31. Viva questions: contribution, future work and honest boundaries 40
32. A small glossary for reading the code and thesis 41
33. How to learn this without memorising a script 43
34. The short revision page 44
35. Sources and coverage 45

1. Start with the problem

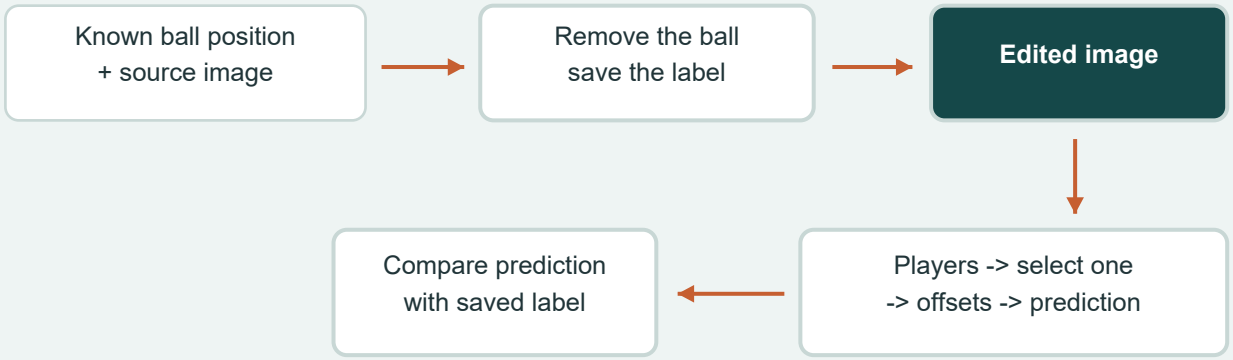
An ordinary ball detector looks for the appearance of a ball. This project asks what can be inferred when that appearance has been removed. The input is one football image with the ball digitally erased. The output is one estimated horizontal and vertical ball coordinate in that image.

The players remain visible. Their positions and body arrangements may contain clues about where the action is happening. The project tests whether those clues improve a prediction compared with guessing the centre or a typical ball position.

The original location is already labelled, which makes the question measurable. An image is edited, a prediction is made from the edited input, and the prediction is compared with the saved label. The system is trying to recover the original physical ball location. It is not deciding where a ball would look aesthetically plausible.

The research question is: **How useful is information about players for locating a missing football, compared with simple position guesses and a scene-based model?**

ONE IMAGE, THREE DIFFERENT USES OF ITS LABEL



The final predictor receives the edited image and features, without the true ball point.

The final method finds players, describes each with numbers, chooses one player and predicts the ball's displacement from that player's centre. It also reports a second point pulled towards the average training-ball position. An established image-based model, Gaze-LLE, supplies a separate comparison.

The result is mixed in an informative way: the raw method produces many close estimates, but some large mistakes keep its average error high. The evaluation then asks where those mistakes come from. That chain of reasoning is the main viva story.

Be able to explain: what goes in, what comes out, why the ball is removed and how the prediction can still be checked. Model names can follow once those four things are clear.

Source: thesis Chapters 1 and 2.

2. The original idea and the pivot

Development began with a more direct attention-based idea: estimate where individual players are looking, draw widening directional bands from their heads and combine the bands. Areas supported by several players would become possible target locations.

The earlier pipeline detected people, associated heads with them and used face crops. L2CS-Net estimated gaze angles; 6DRepNet estimated head orientation. Visibility and image-quality measurements affected each player's contribution to the combined heatmap. This required decisions about matching people, coordinate conventions, which direction to use and how to turn uncertain directions into image scores.

Two problems remained. First, a reliable face-based estimate is difficult when faces are small, tilted or partly hidden. Second, a direction does not by itself tell how far away the target is. A line drawn from a face can pass through many possible locations. Head orientation is also only a rough guide to eye gaze because the eyes can move independently.

The fallback rule was specific. L2CS was normally used. If its unit vector had an on-image component shorter than 0.25, the tool switched to head orientation only if that estimate's on-image component reached 0.25. Otherwise it kept L2CS. When L2CS supplied no estimate, available head orientation was used. This is a projection-length rule, not a claim that a prediction below 0.25 confidence is unreliable.

The final thesis took a different route: **learn the ball's two-dimensional position relative to a player directly**, using player position and body measurements. This avoids requiring an accurate gaze direction for every image. It also changes the claim: final results concern missing-ball localisation, not verified eye-gaze accuracy.

The earlier heatmap combination is reported as supporting work: the top-scoring 10% of image area contained the ball in 39/61 cases for the combined map, versus 46/61 for Gaze-LLE alone. That particular combination did not help.

Later face, iris, UniGaze and pasted-ball experiments are separate exploration. Their screenshots or scores must be identified as later work if used in the viva. Synthetic-face work can explain development difficulties, but the submitted thesis does not quantify a synthetic-training experiment. Do not present it as the experiment that established the final thesis conclusion.

Source: thesis Sections 1.4, 4.2 and 5.6.4. The current repository's deprecated folder is not a complete archive of that gaze pipeline.

3. The data and the split

The local SoccerNet-v3D collection contains **4,051 still images from 379 football matches**. Each has a labelled box around the ball. Its centre supplies the target coordinate. The labels identify the physical ball; they do not identify possession, useful player roles or where each person is looking.

Use	Images	Matches	What it supplies
Training	3,238	318	Examples for fitting the local models
Original evaluation	61	61	One selected image per withheld match
Additional evaluation	752	Same 61	More views of those matches
Complete expanded evaluation	813	Same 61	Original 61 plus the additional 752



The original image per match is selected using a fixed identifier-based rule, rather than by inspecting errors, faces or ball positions. Every image from those evaluation matches is withheld from the current fitting stage. The expanded set contains 810 source-test images and three source-training images, but all 813 belong to this study's withheld matches. The study's split is determined by match membership.

Whole-match separation matters because frames from one match can share players, backgrounds, camera views and nearby moments. Splitting individual frames could put very similar situations on both sides of the comparison.

There is still a limitation: **all 379 matches had been encountered during earlier development**. The current models were fitted without the evaluation matches, but earlier choices of approach and settings could have been influenced by the collection. This is an exploratory evaluation, not a final untouched external test.

The extra 752 images improve coverage of the same matches. They do not provide 752 new independent matches. Two groups of repeated source images were found in training; none crossed into evaluation.

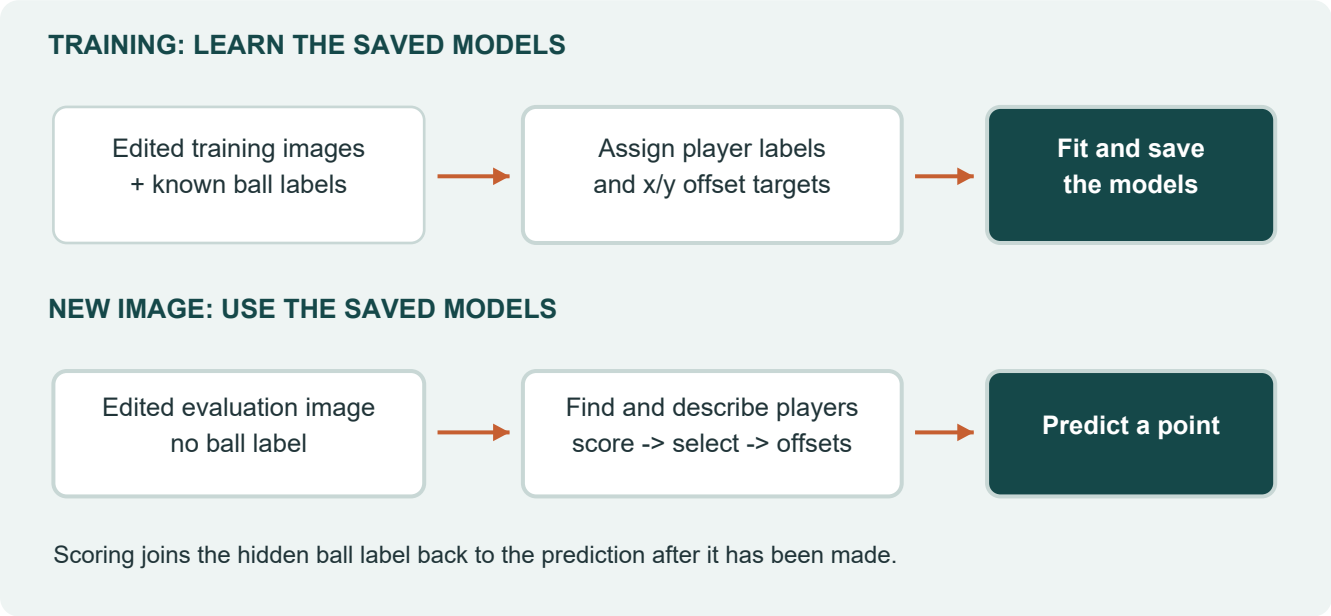
Useful viva wording: "The current fit excludes every evaluation match. However, the collection had been used during development, so a new external set would still be needed to confirm generalisation."

Source: thesis Sections 2.1-2.2 and 5.2.2.

4. Removing the ball without giving away the label

Preparation reads the ball box, rounds its edges outwards, adds two pixels of padding and clips the region to the image boundary. OpenCV's Telea inpainting fills that region using surrounding pixels, with radius three. Training and evaluation follow the same editing rule.

Using the label to make the input is legitimate here: the study needs a ball whose original position can be scored. The important boundary is what reaches ordinary prediction afterwards.



In the current code, labelled records contain fields such as training_target_xy. The function copy_records_without_ball_labels removes ball-label and removal-box fields before detection and ordinary prediction. The prepared image path and dimensions remain. Scoring joins the predicted point to the label again using the image identifier.

Fitting changes model parameters using training examples. Ordinary prediction uses the saved parameters without updating them. A new image does not trigger fresh training on its hidden answer.

Keeping a label out of prediction does not guarantee that the edited pixels reveal nothing about it. Inpainting can leave a trace or alter nearby content. The extra-patch experiment later in the guide tests one aspect of that possibility. Natural occlusion remains a different condition.

The thesis reports that saved inputs matched the prepared pixels and that pixels outside the removal mask stayed unchanged. It also reports two corrected earlier input problems: the grass filter had reopened an unedited image, and detection had received the colour channels in the wrong order. These were corrected before producing the published player-based comparison.

Be able to distinguish: using labels to prepare data, using labels to learn a relationship, using labels to score it, and accidentally using labels to make a test prediction. Only the last would give ordinary prediction information it should not have.

Source: thesis Sections 2.2, 4.3 and 5.6.1; prepare_images.py and main.py.

5. What YOLO supplies, and why the grass check exists

Pretrained YOLOv8m-pose supplies a person box, a detection score and 17 estimated landmarks per person. The landmarks cover the nose, eyes, ears, shoulders, elbows, wrists, hips, knees and ankles. They describe visible arrangement, not a labelled action or an eye-gaze vector.

The thesis detector uses the whole image at input size 1,280 and four overlapping sections at 640, with 15% overlap. Local sections can expose smaller players more clearly, but repeated detections need merging. Duplicate boxes are merged using intersection-over-union or containment rules; detections cut off at tile edges receive a lower score. These are processing sizes, not the dimensions of every source image.

The uploaded simplified detector currently uses only the whole-image pass. This is a real implementation difference, explained in Section 22. The rest of this section explains the purpose of detection in either route.

YOLO detects people, which can include spectators. The field-support filter checks a patch around the bottom of each box for green pixels. The patch uses the middle 60% of the box width, starting 8% of box height above the bottom and ending 12% below it. A pixel passes when green exceeds 45 and exceeds both red and blue by a factor of 1.08. A detection remains when at least 15% of the checked pixels pass.

That is a rough way to favour people on the pitch. It does not prove that someone is a player: a referee may pass and a player may fail because of shadows, kit, a different background or a poor box. The value becomes one of the model's numerical features as well as a filtering rule.

OpenCV reads colour images in BGR order. The grass calculation explicitly converts to RGB to keep the channel names consistent. With this symmetric green rule, swapping only red and blue leaves the grass mask unchanged. Correct input colour order still matters for the learned detector.

Be able to explain: detection proposes people; filtering removes some proposals; landmark extraction describes the retained people. None of these stages knows the missing ball's true position during prediction.

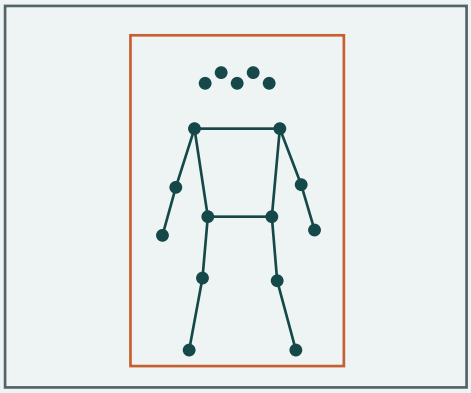
Source: thesis Sections 4.4 and A.1; player_helpers/detect_players.py.

6. What the body points actually have to do with the ball

The idea is that body arrangement can be associated with the action around a ball. For example, a lifted leg or an upward-facing posture might occur in different situations. These are plausible reasons to test body information. The code does not explicitly label "kicking", "heading" or "watching", and the results do not prove that the fitted models learned those particular concepts.

Instead, every retained player becomes a fixed row of measurements. Fitting learns a relationship between those measurements and the labelled ball across many examples. Evaluation then checks whether the learned relationship helps on the withheld matches.

ILLUSTRATION: THE SAME PLAYER IN TWO COORDINATE SYSTEMS



Whole image

Box location and size

Measured relative to the image

Landmark positions

Measured inside this player's box

60 values describe one retained player

No explicit action or gaze label

Feature group	Number	Meaning
Position and detection	7	Box centre x/y, width, height, area, detection score, grass-support score
Extra summary values	2	Box width-to-height ratio and fraction of accepted landmarks
Landmark values	51	x, y and visibility flag for each of 17 points
Full representation	60	All three groups together

Read one player's feature row

The seven position features locate and size the player relative to the whole image. Landmark coordinates instead describe where each point lies **inside the player's own box**. A shoulder halfway across that box has relative x = 0.5, whether the player stands on the left or right of the image. Overall position remains available through the box centre.

Illustration: take a 1,000 x 1,000 image and a player box whose left, top, right and bottom are (400, 200, 600, 800). The box is 200 pixels wide and 600 high. A shoulder at (450, 350) produces the following measurements:

Measurement	Calculation	Value
Player centre x	500 / 1,000	0.50
Player centre y	500 / 1,000	0.50
Player width	200 / 1,000	0.20
Player height	600 / 1,000	0.60
Player area	(200 x 600) / (1,000 x 1,000)	0.12
Shoulder x inside box	(450 - 400) / 200	0.25
Shoulder y inside box	(350 - 200) / 600	0.25

This row describes a player, not the ball. During fitting, similar numerical descriptions are paired with known player assignments and ball offsets. At prediction time, the learned models use those relationships to score the player and estimate a displacement. Similar poses can accompany different ball locations, so the relationship remains imperfect.

A landmark is used when its confidence is at least 0.25. Otherwise its coordinate values become zero and its visibility flag becomes zero. A valid point can also have coordinate zero, so the flag helps distinguish "missing" from "present at this position". Confidence is a detector output, not a guarantee that the point is correctly located.

The features go into **both player selection and offset prediction**. They can therefore influence which player is used and where the ball is predicted relative to that player. Comparing full and position-only models tests the complete route, not just the final location calculation.

The claim the evidence supports: the fuller feature set helps on the expanded evaluation. Since it adds shape and visibility as well as landmarks, the gain cannot be attributed to body pose alone or to a particular joint.

Source: thesis Sections 4.5 and 5.3; player_helpers/measure_players.py.

7. How the system learns which player to choose

This is easiest to understand by separating **training**, when the ball location is known, from **prediction**, when it is hidden.

During training the dataset has no label saying "this is the useful player". The code creates a practical substitute: find the retained player box nearest the known ball. Distance is zero if the ball point lies inside a box; otherwise it is the shortest distance to the box. Ties use distance to the box centre, then player index.

A training image is used for player-based fitting only when that nearest-box distance is at most **0.05 of the pixel image diagonal**. This keeps the training examples focused on nearby player-ball relationships. The threshold was inherited from development rather than established as an optimum.

In a qualifying image, that nearest player receives label 1 and the other retained players receive label 0. The selector learns to distinguish these rows from their numerical features. Out of 3,238 training images, **1,957 qualify**, supplying **18,540 player rows**. The larger row count is because one image can contain several players.

ILLUSTRATION: LABELLED EXAMPLES TEACH A SCORE

Player A training label 0	**Player B training label 1**	Player C training label 0
New-image score 0.20	New-image score 0.72	New-image score 0.35
	B is selected	

During training: nearest labelled box supplies the positive example

During prediction: choose the highest learned score, without the ball label

The selector is logistic regression. It learns a weight for each feature and combines them into a score for the positive class. Training standardisation puts feature values on comparable scales. Balanced class weighting compensates for having many more negative player examples than positives. These operations are fitted on training data.

At prediction time, every retained player gets a score based on their features. **The highest-scoring player is chosen. The true ball location is not consulted.** The scores are separate positive-class scores; they do not have to add to one across the players. They should not be described as calibrated confidence that the ball estimate will be correct.

Illustration: player A receives 0.20, B receives 0.72 and C receives 0.35. B is selected because its score is highest. Those made-up scores illustrate the rule, not actual project outputs.

The full and position-only models have separate selectors, so they can choose different people. Equal scores use the earliest retained player index. Any nonempty candidate list produces a choice, even if all candidates are unhelpful; there is no low-score rejection rule. If no player remains, the output is the training-ball mean.

The nearest-box label is only a **proximity label**. It does not establish that the player is looking at the ball or is the best reference for prediction. That assumption is one reason the later player-selection checks matter.

Source: thesis Sections 2.3 and 4.6; player_model.py and player_helpers/predict_from_players.py.

8. How the selected player becomes a ball coordinate

The selected player's centre supplies a starting point. Two separate regression models predict how far the ball lies horizontally and vertically from that point. These displacements are the offsets.

During training, an offset target is calculated from a known player-ball pair:

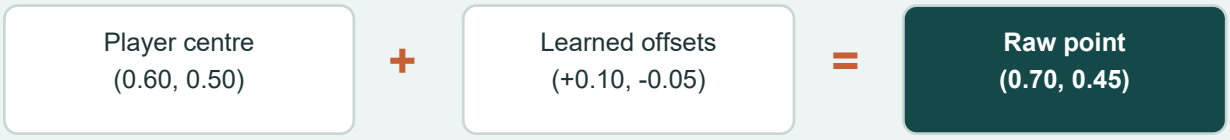
```
horizontal target = labelled ball x - assigned player centre x
vertical target   = labelled ball y - assigned player centre y
```

Only the assigned player in each qualifying image trains the offset models. Each offset model therefore has **1,957 training examples**, whereas the selector uses all 18,540 player rows from those images. The average ball position is calculated separately across all 3,238 training images.

The offset models use histogram-based gradient-boosted regression trees. A tree learns splits on feature values; boosting fits successive trees to improve the remaining errors. Histogram binning makes split finding efficient. The project uses scikit-learn's implementation, rather than writing a new tree-learning algorithm.

At prediction time, the selected player's features go into the horizontal and vertical models, and their outputs are added to the box centre.

ILLUSTRATION: TWO DISPLACEMENTS PRODUCE ONE POINT



x increases rightwards; y increases downwards

Illustration: a selected player has centre (0.60, 0.50). The models predict horizontal offset +0.10 and vertical offset -0.05. The raw point is (0.70, 0.45). Image y increases downwards, so the negative vertical offset moves upwards.

The displacement targets use coordinates normalised by image width and height. They are not measured in metres, ball speeds or gaze angles. Offsets are relative to the player's centre but do not have to remain inside that player's box. The final raw coordinates are clipped to [0, 1] to stay inside the image.

Choosing a wrong player changes both the starting centre and the input features. A good offset model can therefore still produce a poor point after a bad selection. Conversely, a close selected player can already supply a reasonable centre guess. The centre-control experiment is needed to tell how much the offsets themselves add.

Source: thesis Section 4.7 and Table 5.5; player_model.py.

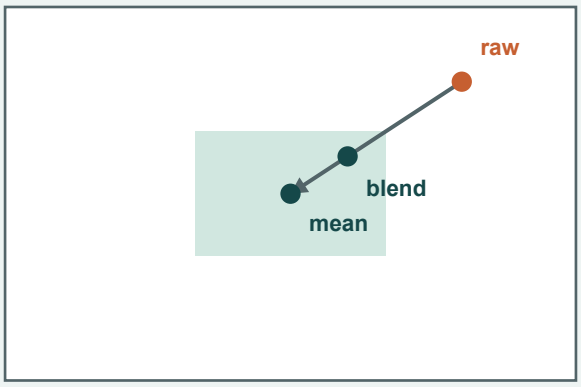
9. Raw prediction, blending and the central rectangle

The **raw prediction** is the selected player's centre plus the two predicted offsets, after clipping. The **full blend** moves that point towards the average ball coordinate in the training images, approximately **(0.4946991, 0.4906810)**.

```
blended point = training mean + (raw point - training mean) / 3
```

This gives one third of the weight to the raw point and two thirds to the training mean. It is not combining predictions from three players or averaging three gaze models.

ILLUSTRATION: EVERY RAW POINT IS PULLED TOWARDS THE MEAN



One third raw

+ two thirds training mean

Green rectangle:

all reachable blended points

1/3 width x 1/3 height = 11.1%

Schematic with mean (0.50, 0.50); exact study bounds are given in the text

Illustration: with a made-up training mean of (0.50, 0.50), raw (0.80, 0.20) becomes blended (0.60, 0.40). That helps if the true ball was central and the raw point was wrong. If the true ball was at (0.80, 0.20), it damages an exact raw prediction. The rule cannot tell those situations apart.

Because raw coordinates are clipped first, the blend can only reach a rectangle one third of the image width and one third of its height. With the recorded training mean, x lies about 0.330-0.663 and y about 0.327-0.660. That is approximately **11.1% of the image area**. A correct raw point outside it is always pulled back towards the middle.

The one-third weight was retained from earlier work and was not tuned using a separate validation set. It was intended to reduce large errors, but the study does not show that it is the best weight. Choosing a new value from the evaluation results would turn those results into tuning data.

The full blend is the original main method in the thesis. Raw results are also reported because they reveal what the blend changes. Always name the output before quoting a score.

When no player remains, raw and blended both return the training mean. There is one such case in the original 61 and 16 across all 813; the latter includes the original fallback. These images remain in the evaluation.

Source: thesis Section 4.8 and Figure 5.2.

10. Why there are seven methods

Each comparison asks a specific question. A fixed-position baseline tests whether looking at the current image contributes anything; a component control tests a choice inside the player-based method.

Method	What it does	Question it helps answer
Full blend	Full features, learned player selection and offsets, then blending	How does the main system perform?
Raw prediction	Same full prediction before blending	What does pulling towards the training mean change?
Position-only blend	Separately fitted selector and offsets using seven features, then blending	Do the extra measurements add value?
Shuffled-description blend	Full models with extra player measurements reassigned, then blending	Does it matter that the extra measurements belong to the right player?
Training-position mean	Same training-ball average for every image	Is a typical location already competitive?
Image centre	Always (0.5, 0.5)	Is the system better than the simplest central guess?
Gaze-LLE	Peak of its unmarked scene heatmap	How does an established image-based model compare?

The shuffle keeps the seven position features attached to each original player. It moves box aspect ratio, landmark visibility and landmark coordinates together between players within that image. Both selection and offset prediction receive these changed descriptions. It does not move people in the image, shuffle ball labels or refit the full model.

A permutation can leave some rows unchanged; with one player it must do so. It can also create inconsistent descriptions, such as a retained width and height alongside another player's aspect ratio. The separately fitted position-only model is therefore the cleaner comparison of feature sets.

Centre and the training mean matter because football broadcasts can frame the action near the middle. The recorded training mean is itself close to centre. A plausible-looking prediction is not sufficient evidence that a model uses the players well.

Source: thesis Sections 2.3, 4.9 and 5.3.

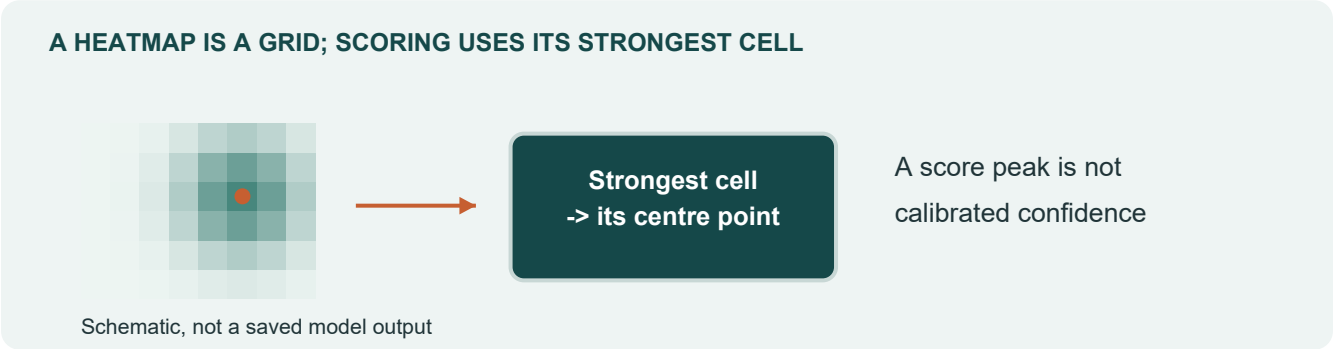
11. What Gaze-LLE is doing in this study

Gaze-LLE is a pretrained gaze-target model. It uses DINOv2 image features and a decoder that scores locations in an image. Its usual person-conditioned use supplies a head marker identifying whose gaze is being estimated.

The main thesis comparison supplies **no head marker**. The edited scene goes into the official supported unmarked setting and the strongest heatmap location becomes a ball proposal. This avoids requiring a separate successful head-selection stage. It is a comparison based on the whole image, not a verified measurement of a particular player's gaze.

The checkpoint uses a DINOv2 ViT-B/14 backbone, was trained on GazeFollow and fine-tuned on VideoAttentionTarget. It was not fine-tuned on this football collection. YOLO and Gaze-LLE keep pretrained weights; the local player selectors and offset regressors are fitted using this study's training set.

Images are resized to 448 x 448 for Gaze-LLE. Its native 64 x 64 heatmap is resized to 128 x 128 for scoring. The strongest cell's **centre** supplies the point. The first maximum in row order settles ties. The checkpoint can also return an in/out output, but the current code keeps its heatmaps and does not use an in/out threshold to reject images from point scoring.



A heatmap is a field of scores. Its brightest location is only one summary of it. Normalising the scores does not demonstrate that a region has a calibrated probability of containing the ball. This thesis's main comparison uses point error, rather than a claimed confidence region.

Older person-marked outputs exist for 49 of the original images. Those maps are normalised per person, averaged and converted to one point. That is a separate matched comparison, described in Section 20. The current simplified head finder can change which images have usable head boxes, so it cannot automatically inherit the historical denominator of 49.

Useful viva wording: "Gaze-LLE is used as a scene-based location baseline. Its main output here has no supplied person marker, so it should not be interpreted as the measured gaze of one selected player."

Source: thesis Sections 1.2, 3.4, 5.6.3 and A.1; gaze_comparison.py and prepare_heatmaps.py.

12. Understanding error and the close threshold

The ball and predicted point are both expressed as fractions of image width and height. Top left is (0, 0), and bottom right is (1, 1). The error is the straight-line distance in that normalised coordinate system:

```
error = sqrt((predicted x - true x)^2 + (predicted y - true y)^2)
close = error <= 0.10
```

Illustration: a horizontal difference of 0.06 and a vertical difference of 0.08 give error 0.10. On a 1,920 x 1,080 image, a purely horizontal 192-pixel miss or purely vertical 108-pixel miss reaches the threshold.

This is a study-defined threshold for nearby estimates, not a confidence level, ball-radius criterion or universal measure of practical usefulness. Because the axes are normalised separately, it is also **not 10% of the pixel image diagonal**. With points inside the image, error can reach sqrt(2), about 1.414; it is not limited to one.

Distance used	How it is measured	Where it is used
Ball-to-player-box distance	Shortest pixel distance to box, divided by pixel diagonal	0.05 training eligibility and nearest-box labels
Ball-prediction error	x divided by width, y by height, then Euclidean distance	Main scores and 0.10 close threshold
Ball-to-extra-patch distance	Separately normalised x/y distance	Minimum 0.20 for the extra edit

The **mean** includes every error and responds strongly to large misses. The **median** is the middle error. The **close count** records how many pass the chosen threshold. None alone tells the complete story.

Illustration: one method has errors 0.02, 0.02, 0.02, 0.02 and 0.80. Four are close, but mean error is 0.176. Another has five errors of 0.11: none is close, but its mean is better at 0.11. These made-up lists explain how more close results can coexist with a worse average.

In Figure 5.1, the horizontal axis is an allowed error and the vertical axis is the fraction of predictions at or below it. Higher at the same error means more predictions are within that distance. Read the height at 0.10 for close rate, and inspect the rest of the curve for the remaining errors.

Source: thesis Section 2.4 and Figure 5.1.

13. Why matches are weighted and resampled

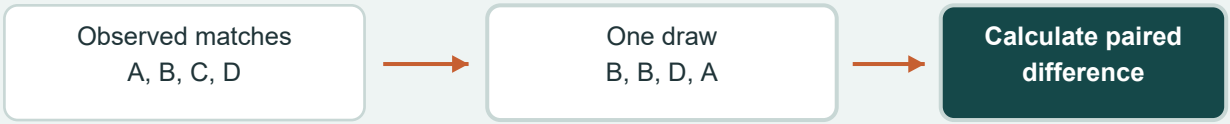
The expanded evaluation has different numbers of images per match. An average across all images gives more influence to matches with more frames. The main expanded result instead calculates mean error and close rate inside each match, then averages the 61 match results. This gives every match equal influence.

Illustration: match A has one image with error 0.10; match B has nine images each with error 0.30. The image average is 0.28, but the equal-match average is (0.10 + 0.30) / 2 = 0.20. Neither is arithmetically wrong; they summarise different units.

The expanded raw close count is **271/813**, roughly 33.3% of images, while its equal-match close rate is **32.3%**. That difference comes from weighting. The main table deliberately reports match-weighted rates and actual image counts in separate columns.

The paired bootstrap describes uncertainty in a method difference across the observed matches. Each of **5,000** draws samples whole matches with replacement. A selected match brings its frames with it. Both methods receive the same sampled matches, so their difference preserves the pairing.

ILLUSTRATION: RESAMPLE MATCHES, KEEP THE METHODS PAIRED



Repeat 5,000 times; take the 2.5th and 97.5th percentiles

Frames stay with their match; both methods use the same draw

For each draw, the analysis calculates the difference. The 2.5th and 97.5th percentiles of those differences form a pointwise 95% percentile interval. The recorded procedure uses NumPy RandomState seed 20260721 and linear percentile interpolation.

For error, **first method minus comparator** below zero favours the first method. If the interval crosses zero, the comparison does not clearly establish the direction of improvement. This does not prove the methods equivalent. For close-rate differences, positive values favour the first method.

These are individual, exploratory intervals. They are not a guarantee that all comparisons are correct simultaneously. They hold the developed approach and settings fixed, and they do not remove earlier data exposure or create new matches.

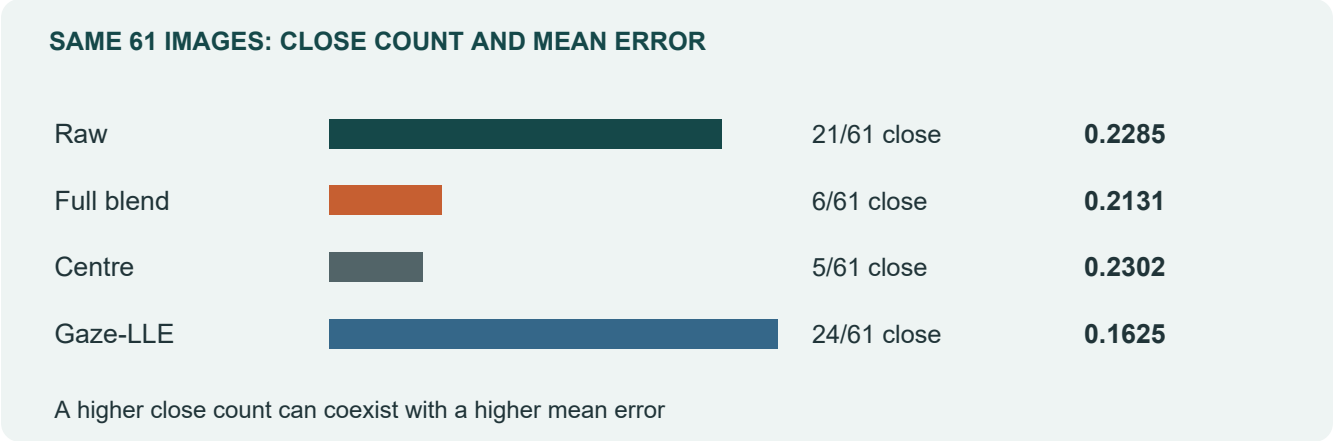
Percentage-point differences subtract rates. A rise from 10% to 15% is five percentage points, not a five-percent relative increase. The thesis uses pp in the appendix tables.

Source: thesis Sections 2.6 and 5.2.2; calculate_results.py.

14. The original 61-image result

Every method is scored on the same 61 images, including the no-player fallback. The full blend is the main method; raw results expose its trade-off.

Method	Mean error	Median	Close
Full blend	0.2131	0.2012	6/61
Raw prediction	0.2285	0.1681	21/61
Position-only blend	0.2132	0.2076	7/61
Shuffled-description blend	0.2227	0.2186	6/61
Training-position mean	0.2291	0.2334	4/61
Image centre	0.2302	0.2307	5/61
Gaze-LLE, no supplied head	0.1625	0.1284	24/61



The raw model makes **21 close predictions versus five for centre**. Its lower median also shows that it can get substantially closer in some images. However, large remaining mistakes leave its mean near centre.

Blending lowers the reported mean from 0.2285 to 0.2131 but reduces close predictions from 21 to six. The original data do not clearly establish the blend's mean-error improvement over the fixed-position baselines: full blend minus centre is -0.01710 with interval [-0.03539, +0.00140].

The raw close-rate gain over centre is supported: +26.23 percentage points, interval [+13.11, +39.34]. Its mean-error difference from centre remains unclear: -0.00176, interval [-0.05316, +0.04962]. This is why "no improvement over baselines" is too broad.

Gaze-LLE has the best mean, median and close count in this table. Raw has three fewer close predictions, but the close-rate difference is uncertain. Gaze-LLE's lower mean is the clearer gap.

Useful viva wording: "Raw prediction often gets close, but its large misses matter. The fixed blend reduces some of that cost while losing close estimates, and Gaze-LLE has lower mean error."

Source: thesis Table 5.1 and Appendix Tables A.1 and A.3.

15. The 813-image result and the feature evidence

The same fitted models are used on all available images from the withheld matches. The 61 original images remain included. No new independent matches are added.

Method	Match mean error	Match close rate	Image close count
Full blend	0.2125	15.6%	125/813
Raw prediction	0.2294	32.3%	271/813
Position-only blend	0.2238	12.9%	108/813
Shuffled-description blend	0.2360	12.0%	98/813
Training-position mean	0.2260	10.5%	87/813
Image centre	0.2267	10.3%	87/813
Gaze-LLE	0.1909	34.7%	288/813

The larger comparison supports a modest mean-error gain for the full blend over centre: -0.01415, interval [-0.01964, -0.00869]. It also supports a gain over the position-only blend: -0.01130, interval [-0.01562, -0.00679].

On the original sample, full and position-only blends have almost identical mean errors, 0.2131 and 0.2132, and their difference is unclear. On the expanded sample the fuller feature route does better than both the position-only and shuffled routes. That supports some value from the extra measurements under this evaluation, without isolating any one body point.

Raw gives **271 close estimates versus 87 for centre**, while Gaze-LLE gives 288. Raw's image close rate is about 33.3%, and Gaze-LLE's about 35.4%. These are fairly close on that particular measure; the mean-error gap, **0.2294 versus 0.1909**, remains substantial. The table alone does not establish statistical equivalence of close rates.

Do not describe 271 versus 87 as "three times more accurate overall". It is a comparison of one threshold count for the raw output. The main full blend has 125 close estimates, and raw does not beat Gaze-LLE overall.

What this adds to the story: the same broad behaviour appears across more views. The project has useful local estimates, while player choice, distant-ball cases and large misses remain the important weaknesses.

Source: thesis Tables 5.2 and A.2; Sections 5.3-5.4.

16. Why some images are much harder

The offset models were trained only when a retained player box was near the labelled ball. Prediction nevertheless runs on every evaluation image. The eligibility check asks how the system behaves when the training condition is present or absent.

Using the known labels after prediction, the original 61 images divide into **37 eligible** and **24 ineligible** cases. The ineligible group includes the no-player fallback.

Method	Eligible mean	Eligible close	Ineligible mean	Ineligible close
Raw prediction	0.1465	20/37	0.3548	1/24
Full blend	0.1796	4/37	0.2648	2/24
Gaze-LLE	0.1428	17/37	0.1929	7/24

Raw performs much better in the near-player group. The ball can be far from every player in the other group, which is outside the local relationship enforced during training. Other differences between the groups may also affect difficulty, so this is evidence of a coverage gap rather than a controlled proof that distance alone causes the failure.

Blending makes average error worse in the eligible group and better in the ineligible group. That helps explain how it can reduce some large errors while destroying useful close estimates.

Eligibility is not an available rejection rule for an unknown input: it needs the true ball coordinate. Dropping ineligible images from the main result would hide difficult cases using information ordinary prediction does not have.

Useful viva wording: "The model learns local player-ball offsets, but it has to predict even when the ball is far from every player. The diagnostic shows that this mismatch is one important source of large errors."

Source: thesis Table 5.3 and Section 5.5.1.

17. Answer-assisted selection, without the confusing name

Answer-assisted means **selection using the ground-truth ball position**. It changes the player choice to the nearest retained box under the same proximity rule used for training. The full offset models, clipping, blend and evaluation images stay unchanged.

This does not try every predicted point and keep whichever is nearest the answer. It chooses by box proximity, then applies the existing location models.

Selection and point rule	Mean error	Median	Close
Actual selector, raw	0.2285	0.1681	21/61
Answer-assisted, raw	0.1029	0.0796	34/61
Actual selector, blend	0.2131	0.2012	6/61
Answer-assisted, blend	0.1740	0.1634	8/61

The actual selector agrees with that nearest-box choice in **26 of 60 images with players**, including **22 of the 37 eligible images**. These are agreement counts with a proximity rule, not measured gaze-selection accuracy.

Changing the player sharply improves raw performance. Its paired mean-error difference is -0.12560, interval [-0.17006, -0.08295]. This points to selection as a major source of error.

There is an essential comparison beside that improvement: simply returning the answer-selected player's centre gives **0.1150 mean error and 34 close predictions**. The offsets reduce mean error to **0.1029**, with the **same 34 close count**. Much of the answer-assisted success comes from being handed a player near the ball.

That smaller offset gain still matters, but it is not evidence that the regressor alone solved the problem. Changing a player changes both the starting centre and its feature vector. The diagnostic measures how selection and offsets work together.

This is not a deployable predictor or a strict upper bound on every possible method: it needs the ball label, and the nearest player need not be the most informative one. Its value is in locating a weakness of the actual route.

Source: thesis Table 5.4 and Appendix Table A.4.

18. What did the two offset models earn?

This check holds the full selector's chosen player fixed and compares its predicted ball point with that player's bare centre. It asks whether learning a displacement adds anything beyond choosing a person.

Point rule	Original mean	Original close	Expanded match mean	Expanded match close
Full raw	0.2285	21/61	0.2294	32.3%
Position-only raw	0.2243	20/61	0.2693	24.2%
Full selector's player centre	0.2417	23/61	0.2441	27.3%

On the original sample, offsets improve mean error compared with the chosen centre, but produce two fewer close estimates. On the expanded sample, both mean error and close rate improve. This is a modest, specific contribution rather than a large standalone gain.

The original full-raw minus chosen-centre mean difference is **-0.01324**, interval **[-0.02184, -0.00469]**. For the expanded match-weighted result it is **-0.01465**, interval **[-0.01713, -0.01219]**.

Under answer-assisted selection, offsets improve mean error by **-0.01213**, interval **[-0.01857, -0.00576]**, while both centre and offset rules have 34 close predictions. Equal total counts do not mean exactly the same images are close under both rules.

Full versus position-only raw does not hold the selected player fixed. Those selectors choose different players in **341 expanded images**, so that comparison includes both selection and displacement prediction. Original mean difference is unclear; expanded mean error favours the full route.

The appendix recovers historical position-only raw coordinates by reversing the saved blend, then checks reblending within 10^-12. That is an analysis of saved coordinates, not another training run. In current code, raw points are produced during prediction and available directly as well.

Useful viva wording: "Offsets add a smaller improvement beyond the selected player's centre. The much larger answer-assisted change comes from selecting a more useful reference player."

Source: thesis Table 5.5 and Appendix Tables A.6-A.7.

19. The extra edit and replay checks

An inpainting trace could be an unintended clue. The extra-patch check adds another edit with the same mask size and inpainting procedure at a distant location. Its centre must be at least 0.20 from the ball in separately normalised coordinates, and the two masks must not overlap.

First, the unmodified prepared inputs are rerun. All 61 saved native Gaze-LLE heatmaps reproduce exactly. This makes it easier to interpret changes after adding the patch, rather than confusing them with a difference in the baseline run.

All 61 extra patches change pixels. **Sixty predicted points stay unchanged, one moves away from the new patch and none move towards it.** Close predictions stay at 24. Mean error changes from **0.16254 to 0.16217**.

The result gives no evidence that the added patch attracts the strongest predicted point in these inputs. It does not prove that the original ball-removal trace was irrelevant. A heatmap can also change without moving its maximum. Eight additional masks overlap player boxes, with no manual determination of exactly what meaningful content was removed.

The published replay checks also reproduce saved player points in both evaluation samples. Replaying an output is a consistency check. It does not establish that the output is accurate or the method generalises.

Keep this experiment separate from later experiments that paste a visible ball into images. An extra inpainted patch asks about response to an edit; a pasted ball asks about response to a newly visible object. They change different things.

Source: thesis Sections 5.6.1-5.6.2, 5.8 and A.3.

20. The 49-image and earlier heatmap checks

Historical person-marked Gaze-LLE outputs are available for 49 of the original images. Each person's map is normalised, the maps are averaged and their strongest point is scored. All methods must use those same 49 images for a fair matched comparison.

Method	Mean	Median	Close
Full blend	0.2194	0.2002	3/49
Raw	0.2262	0.1664	17/49
Position-only blend	0.2166	0.2076	5/49
Shuffled-description blend	0.2316	0.2216	4/49
Training-position mean	0.2345	0.2380	2/49
Centre	0.2357	0.2316	2/49
Gaze-LLE, unmarked	0.1700	0.1358	17/49
Gaze-LLE, person-marked average	0.1811	0.1230	17/49

Unmarked Gaze-LLE has slightly lower observed mean error than the person-marked average, while both have 17 close estimates. This does not establish that head markers are generally harmful. The result concerns this checkpoint, aggregation, sample and task. Twelve other original images lacked usable historical head outputs.

The earlier head-and-eye combination is a separate measurement again. It multiplies a direction-based heatmap with Gaze-LLE's heatmap, then compares **ball coverage inside the top-scoring 10% of image area**. The combination covers the ball in 39/61 images versus 46/61 for the scene map alone.

Those are **region-coverage counts**, not predictions within point distance 0.10. They cannot be compared directly with the 24/61 close-point count. The tested multiplication did not improve region coverage; it does not prove that every possible use of head or eye information must fail.

Source: thesis Sections 5.6.3-5.6.4 and Appendix A.4, Table A.5.

21. What the literature contributes to the argument

The literature review is not a claim that no one had used players, faces or shared attention before. It locates this implementation among established ideas and explains which evidence can and cannot transfer to the study. The numbers in different papers are not scores on this football comparison.

Thesis reference	Main idea to understand	Connection to this project
[1] Spot the Ball	Infer a missing ball from what remains in an image	Closest task motivation; centre and wrong-player biases matter
[7] Maksai et al.	Ball-player interactions, trajectories and physical constraints	Player information can help, but calibrated video adds evidence absent from one still
[8] TrackNet	Predict ball heatmaps from pixels and nearby frames	Motivates a direct image-to-ball baseline and the value of motion
[9] SoccerNet-Tracking	Tracking people and ball in football video	Documents small, fast or hidden-ball difficulties
[10] L2CS-Net	Estimate gaze yaw and pitch from a face image	Provides an angle estimate for the earlier route, not a complete target distance
[11] 6DRepNet	Estimate three-dimensional head rotation	Head facing direction can differ from eye gaze
[12] Gaze360	Labelled 3D gaze across varied conditions	Relevant gaze data and training background; different inputs still require local testing
[13] GazeFollow	Estimate a person's gaze target from head and scene	Explains the head marker's role
[2] Gaze-LLE	Pretrained visual encoder with a gaze decoder	Established comparison; unmarked use supplies a scene proposal
[14] Harari et al.	Shared attention using gaze and depth	Several people can support a common target, with geometry assumptions
[15] Nakatani et al.	Use relationships, actions and gaze for joint attention	Relevant to future multi-player reasoning; not a new untouched research area

Data, tools and evaluation practice

Thesis reference	Main idea to understand	Connection to this project
[16] Ultralytics pose	Pretrained person boxes and landmarks	Supplies features; landmark positions are not measured gaze
[17]-[18] scikit-learn	Standardisation, logistic regression and boosted trees	Supplies the learning algorithms fitted locally
[4] SoccerNet-v3D	Broadcast images, ball labels and scene geometry	Source of physical ball labels; this study uses 2D positions
[5] Telea	Fill a masked area using nearby content	Creates the missing-ball inputs
[19] Geirhos et al.	Models may exploit shortcuts	Motivates checking for edit-related cues
[20] Lapuschkin et al.	Inspect predictions under controlled input changes	Supports intervention-style checks without proving every underlying cause
[21] Bylinskii et al.	Different metrics reveal different behaviour	Supports interpreting mean, median and close rate together
[3] Kapoor and Narayanan	Leakage weakens ML conclusions	Explains match separation and the remaining development-exposure limitation
[6] Efron	Bootstrap resampling	Background for uncertainty in paired comparisons

For the viva, be ready to explain the closest alternatives: missing-object inference, tracking with time information, face-based directions, scene-based gaze targets and joint attention. Then say which information this project actually uses. That is more useful than reciting every author and publication year.

Source: thesis Chapter 3 and its bibliography, references [1]-[21].

22. The uploaded code and the submitted thesis are not identical

The repository checked is https://github.com/Juice234/Quorum-Gaze at commit **2c57c02fda176855126c4f883fb527cbc2151481**. It contains the submitted Thesis.pdf and a simplified implementation of the main player-based idea. The source PDF exactly matches the supplied local submitted thesis.

Part	Submitted thesis	Current uploaded implementation
Player detection	Whole image at 1,280 plus four sections at 640, followed by duplicate merging	One whole-image pass at 1,280
Main learning route	Player features, selection, x/y offsets, clipping and fixed blend	Same central route
Optional head preparation	Historical outputs from the earlier preparation	Boxes constructed from saved YOLO facial landmarks
Historical matched head sample	49 available original outputs	Availability follows current detections; may differ
Earlier direction heatmap experiment	Described as supporting work	No complete L2CS/6DRepNet pipeline in the current active or deprecated code
Raw position-only analysis	Historical coordinates recovered from saved blends	Current predictor also saves raw points directly

Detection changes can alter accepted players, their features, selected players and fitted models. Loading the same model bundle does not guarantee the same predictions if its inputs change. Matching the basic pipeline is therefore different from reproducing the published values exactly.

This guide does not assign the thesis tables to a newly executed run of the simplified code. No new reproduction was performed while writing it. Historical verified code and saved result records exist separately in the project workspace; those are the appropriate starting point for an exact historical reconstruction, not an assumption that the current GitHub tree is identical.

The **deprecated/** folder contains an older scraper and a Tkinter/YOLO application with simple head-crop geometry. It is not a complete archive of all the original gaze engineering. The root name Quorum-Gaze and project name Football Gaze do not change the final thesis's actual output into measured gaze.

If asked directly: "The uploaded code is a simplified version of the method. The thesis used additional tiled detection, and simplifying that can change results. The published tables come from the recorded thesis implementation, not a claim that this shortened tree is byte-for-byte the same pipeline."

Source: direct comparison of the uploaded code with thesis Sections 4.4, 5.6.3 and A.1. This is a code/document distinction, not a new scientific result.

23. What every current code file is for

There are **18 active Python files**. Start with main.py and follow the six stages. Supporting analysis is deliberately separate from the ordinary prediction route.

File	What it takes and does	What comes out
main.py	Reads requested stage and paths, saves settings, dispatches work	prepare -> detect -> fit -> predict -> gaze -> score
prepare_images.py	Reads annotations, selects frames and matches, removes balls, withholds labels for inference	Edited PNG images and records.json
player_model.py	Connects saved records to detection, fitting and prediction	Detections, models.joblib, model_record.json and player_points.json
player_helpers/detect_players.py	Runs whole-image YOLO, reads its output, measures grass support and point-to-box distance	Retained player boxes, keypoints and scores
player_helpers/measure_players.py	Converts each player into 7/60 features; finds nearest labelled box; creates per-image shuffle	Feature rows and training/diagnostic association
player_helpers/predict_from_players.py	Scores candidates, chooses a player, predicts offsets and applies clipping/blend	Raw, blended and control points
gaze_comparison.py	Loads local Gaze-LLE/DINOv2 and runs image inputs in requested mode	Native heatmaps and point records
prepare_heatmaps.py	Resizes and normalises maps and averages person maps	Common 128-grid representation
calculate_results.py	Joins labels and predictions, measures errors, groups by match and resamples differences	Main summaries, per-image and per-match results
graphs.py	Reads calculated scores and coordinates	Error-distribution and coordinate-example plots

Supporting analysis and shared files

File	Role	Why it exists
supporting_analysis.py	Runs players, heads or extra_patch checks	Keeps label-assisted diagnostics outside ordinary prediction
supporting_checks/player_checks.py	Eligibility, answer-assisted predictions and player-centre controls	Explains selection and offset contributions
supporting_checks/extra_patch.py	Creates distant edits and compares original/edited predictions	Investigates response to another inpainting trace
head_detection/find_head_boxes.py	Builds boxes from at least two confident facial landmarks on a player	Optional person-marked Gaze-LLE input
study_settings.py	Stores numerical rules, seeds and method names	Common values used across files
file_helpers.py	Reads and writes JSON	Shared file operations
download_data.py	Obtains annotations and requested match archives	Dataset inputs
download_models.py	Obtains pretrained weights and recorded third-party source versions	external/assets.json with local model/source paths

The **README.md** explains setup and commands. **requirements.txt** lists package versions. **.gitignore** excludes data, model/runtime outputs and local environment files from Git. **.gitattributes** supplies Git file handling. **docs/** contains a saved comparison illustration, small_comparison.png. **Demo2C.mp4** is a recording, not an input needed by the method. **Thesis.pdf** is the submitted document. None of these is a fitted player model.

Inside **deprecated/**, run_gui.py starts the old interface; src/quorum/gui.py handles its window and controls; player_detector.py wraps the older detector; gaze/geometry.py contains crop/geometry helpers. The quorum_scraper files organise download records, export and scraper/database utilities; its tests cover that scraper. The package initialisation files and pyproject.toml belong to that older package layout. These files are outside main.py's football workflow and are not needed to understand the final training calculation.

Source: all current repository code and text files, read at the recorded commit. The video was inventoried as a recording; it is not evidence for a thesis metric.

24. Follow one image and one player through the code

Start with **main.py -> run_main_stage**. The preparation stage reads the source labels. For other stages, records are loaded, then copies without the ball fields are made. Fit receives training records with labels; ordinary prediction receives evaluation copies without them.

Stage	Function or file to find	What to look for
prepare	prepare_ball_removed_images in prepare_images.py	Source image identity, match split, removal mask and prepared path
detect	detect_players_in_study_images in player_model.py	Current whole-image detection and grass filtering
features	calculate_player_features in measure_players.py	Box coordinates and each landmark's relative position/visibility
fit	fit_player_selection_and_offset_models in player_model.py	Positive nearest-player labels and separate x/y offset targets
predict	predict_ball_from_player_features in predict_from_players.py	predict_proba, argmax, selected feature row, offset predictions, clipping
gaze	predict_with_gaze_lle in gaze_comparison.py	no_head mode, native maps and converted points
score	calculate_main_results in calculate_results.py	Label join, errors, close rule, match grouping and saved summaries

The key line for player choice is the maximum positive-class score after selector.predict_proba. The key location calculation adds each regressor's output to the selected box centre. The blend helper then moves that point towards the saved training average. Those three decisions are separate.

models.joblib holds six fitted estimators: a full selector and two full offsets, plus a position-only selector and two position-only offsets, together with the training mean. The selector objects include their fitted standardisation. The bundle stores learned model objects, not just a list of final thesis scores.

The compatibility loader translates older saved names, including prior to training_average and full_xy/position_xy to the offset-model names. It does not refit those objects. A compatible file format says the objects can load; it does not demonstrate identical features or results.

The Gaze-LLE loader temporarily replaces a backbone-loading function with an already loaded local DINOv2 backbone. Its use of unittest.mock.patch does not mean fake predictions are being generated. It is a loading hook to use local third-party assets.

A source row describes **one image**. A detection list describes **several people in that image**. A feature-table row describes **one person**. A predicted point describes **one method's output for one image**. Keeping those units separate makes the training counts and file contents much easier to follow.

Source: main.py, player_model.py, gaze_comparison.py and player_helpers/.

25. Running the current repository and reading its outputs

These are the current commands, checked against the uploaded argument parsers. They were not executed as a new scientific run for this guide. They run the simplified implementation described above.

From the repository root in PowerShell, create the environment and obtain the model assets:

```
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe download_models.py --destination external
```

PyCharm should use .venv\Scripts\python.exe as its interpreter. The leading .\ tells PowerShell to use an executable relative to the current folder. The environment must exist before that executable can be used.

Obtain annotations and, if needed and authorised, the match-frame archives:

```
.\.venv\Scripts\python.exe download_data.py --annotations data/annotations
.\.venv\Scripts\python.exe download_data.py `
  --annotations data/annotations --frames data/frames
```

The download script's --annotations argument is a destination folder. In main.py it is the path of the downloaded CSV. Source frames require SoccerNet access. Leave Frames-v3.zip inside each match folder; the current reader expects data/frames/<league>/<season>/<match>/Frames-v3.zip and opens the required images inside it.

With the data and assets available, run the main workflow:

```
.\.venv\Scripts\python.exe main.py all `
  --run-dir runs/football `
  --annotations data/annotations/SNv3D.csv `
  --images data/frames `
  --assets external/assets.json
```

The stage order is **prepare -> detect -> fit -> predict -> gaze -> score**. Replace all with an individual stage to run just that step after its prerequisites exist. For example, scoring saved predictions uses:

```
.\.venv\Scripts\python.exe main.py score --run-dir runs/football
```

This recalculates summaries and figures, not detection or model fitting. The optional analyses run separately:

```
.\.venv\Scripts\python.exe supporting_analysis.py players --run-dir runs/football
.\.venv\Scripts\python.exe supporting_analysis.py heads `
  --run-dir runs/football --assets external/assets.json
.\.venv\Scripts\python.exe supporting_analysis.py extra_patch `
  --run-dir runs/football --assets external/assets.json
```

Existing detections, fitted models and predictions may be reused. Use a **new run folder** after changing code, settings or inputs. Writing a fresh settings.json beside old outputs does not make those outputs reflect the new settings. This is an important limitation of simple file-existence reuse.

Read the output files

Output under the run folder	What to open it for
settings.json	Requested processing and analysis values
data/records.json and data/images/	Image identities, split and prepared inputs
detections/	Retained players for an individual image
fit/models.joblib and fit/model_record.json	Fitted models, feature columns and training counts
predictions/player_points.json	Selected player and raw/blended/control points
gaze_lle/no_head/ and no_head_points.json	Gaze-LLE maps and main point predictions
results/original_61_images/ and all_813_images/	Summary and per-image points; expanded per-match metrics
results/player_selection_check/ and player_centre_comparison/	Optional player-based diagnostics
results/head_marker_comparison/ and extra_patch_check/	Optional gaze and editing checks
figures/	errors_61_images.png, errors_813_images.png, prediction_examples.png

An output named all_813_images describes the intended expanded set, not proof that its contents reproduce the submitted table. Check the run's actual input records and implementation. The current source also saves label-flow information in results/data_flow.json.

Source: current README, main.py and supporting_analysis.py.

26. Settings and dependencies without mystery

Some values are study choices; others are library behaviour. A "Library defaults" comment in study_settings.py does not make every value below it a default. Do not invent a tuning study to justify settings that were retained from development.

Setting	Role and what can honestly be said
Removal padding 2, radius 3	Recorded editing configuration applied consistently
Whole-image input 1,280, confidence 0.15	Project detector configuration; source dimensions still vary
Thesis-only tiles 640, overlap 0.15	Additional views in the published implementation, absent from current simplified detection
Thesis duplicate rules: IoU 0.45, containment 0.75, edge factor 0.85	Merge repeated detections and lower tile-edge scores; absent from current simplified detection
Grass support 0.15	Simple retention rule, not a measured probability of being a player
Landmark cutoff 0.25	Decides whether to include a coordinate or mark it missing
Near-ball cutoff 0.05 of pixel diagonal	Inherited rule for training association, not optimised here
Blend one third	Inherited shrinkage choice, not validated as optimal
Close error 0.10	Study reporting convention for point predictions
Gaze grid 128	Resize of native 64-grid heatmaps for scoring
Bootstrap 5,000 draws	Recorded whole-match uncertainty procedure

The selector uses balanced class weights and at most 2,000 iterations. The offset regressors use learning rate **0.05**, at most **250 iterations**, at most **15 leaves**, at least **20 samples per leaf** and L2 regularisation **1.0**. Learning rate controls each boosting update; it has no link to the unrelated 0.05 ball-to-box distance.

The current optional head finder uses at least two of the nose/eye/ear points at confidence 0.3. Its head-size rule uses landmark spread scaled by 2.2, a minimum of 0.06 of player height and a final expansion factor of 1.6. This 0.3 head-box rule differs from the 0.25 landmark cutoff for player features. Neither supplies iris visibility or measured gaze confidence.

The full and position-only horizontal seeds are 20260803 and 20260703; vertical seeds add 100. Selector seed is 20260703. Detection seed is 2026101502. The player shuffle uses seed 20260703 plus an image identifier and retained prefix pivotal. Gaze seed is 20261015, extra-patch seed 2026101503 and bootstrap seed 20260721. A seed fixes a particular random sequence, not a confidence score, date of an experiment or proof of scientific quality.

Packages and repeatability

Package or tool	Its role
Ultralytics and PyTorch	Run pretrained YOLO and gaze neural models
scikit-learn	Fit the selector, standardiser and offset regressors
NumPy and pandas	Coordinate/array calculations and feature tables
OpenCV	Image reading, colour conversion, inpainting and heatmap resizing
joblib	Save and load fitted estimators
Matplotlib	Plot calculated results
SoccerNet/Hugging Face tools	Obtain authorised data and model assets
pathlib, argparse, json, zipfile	Python's built-in path, command, record and archive tools

Eight-thread limits and recorded package versions help numerical repeatability. They do not repair a methodological limitation. Changing packages, detector processing or input preparation can change results even when model filenames remain the same.

The thesis records Python 3.11.9, PyTorch 2.4.1, NumPy 2.4.6, scikit-learn 1.9.0 and OpenCV 4.10. These are the submitted document's environment details; they are not a claim that a fresh current run was validated here.

Source: thesis A.1, requirements.txt and study_settings.py.

27. The contribution, limits and worthwhile next steps

The contribution is the implemented **player-selection and relative-location system**, with comparisons that explain the effects of its features, selection, offsets and blending. YOLO, Gaze-LLE, logistic regression and boosted trees are established components. The project uses and fits them rather than inventing their architectures.

That is a legitimate system and evaluation contribution. It should not be presented as the first use of player information for ball localisation or a new general gaze estimator. The literature already includes ball-player interaction models and joint attention.

The strongest explanation goes beyond "Gaze-LLE won": player information can yield useful close estimates, player choice strongly affects errors, learned offsets add a smaller benefit and a fixed central blend can damage correct predictions. The fuller representation helps in the expanded comparison, although the gain is not isolated to body pose alone.

The principal limits are earlier exposure to the evaluation matches, artificial removal, imperfect nearest-box labels, limited local training coverage, an untuned blend and the lack of a direct image-to-ball model trained on the same data. Not every training image was manually inspected. Preparation and training changed together between implementations, so their separate effects were not measured. The thesis does not demonstrate practical user benefit, generalisation to new sports or accurate individual gaze.

The first next step in the thesis is better supervision for selecting a useful player. Independent role labels would test whether proximity is the right training target. Broader training examples could cover balls far from the players. A match-separated validation set would choose a blend weight without tuning on the final evaluation.

A direct image-to-ball baseline would make the representation comparison closer. Naturally hidden balls, with defensible hidden-location labels, would test transfer beyond editing. Independent new matches should be reserved for a final check. A user study becomes useful when predictions are reliable enough to assess as an annotation aid.

Where the all-player and head-pose idea fits

Using several players could reduce dependence on one poor selection. A future system could assess head orientation and visibility, give uncertain players less influence and seek a location supported by several compatible directions. This is a proposed extension; it was not established by the final player-offset results.

Two people looking towards each other may be attending to one another rather than a ball. That could help identify a competing interaction, but is not automatic ball evidence. Likewise, parallel arrows in the image do not guarantee a common target, and a common target can lie in different directions from different people.

The stronger formulation is **look for agreement on a possible target location, with uncertainty**, rather than require identical head angles. It would need labelled evaluation: not everyone watches the ball, head pose differs from gaze and some faces have too little detail. The existing joint-attention literature is the starting point.

Useful viva wording: "Using all players could make the method less dependent on one reference. A next version could test whether several reliable head or body cues support the same location, while allowing for players who are watching each other or looking elsewhere."

Source: thesis Sections 3.5, 5.7-5.8 and 6.3. The final subsection explains the proposed viva extension, not a completed thesis experiment.

28. Viva questions: explain the project first

These are plain answer starting points. Practise explaining the reason, then give the evidence if asked. Saying what was not tested is appropriate when it limits a claim.

What is the project trying to do?

Predict the original location of a football after it has been removed from a single image, using information left in the players. The saved original label allows the prediction to be measured.

Why is this different from detecting a ball?

The ball's appearance is absent. A visible-ball detector cannot solve that input in the usual way, so the system must infer location from other cues.

What would this be useful for?

A location suggestion might give someone a starting point when checking or annotating a difficult image. The thesis measures localisation; it does not establish that users work faster or better.

What was the original idea, and why change it?

The original route combined estimated looking directions. Reliable facial cues were difficult in these images, and direction alone left target distance unresolved. The final route learned a two-dimensional ball offset from a selected player's measurements.

Did the original approach fail because the models were bad?

That is too broad. The complete pipeline had difficult inputs, preparation and coordinate dependencies as well as model limitations. The tested combination did not improve coverage. The thesis does not isolate a single cause or demonstrate that all face-based methods fail.

Is the final method actually measuring gaze?

It estimates a ball coordinate from player features. There are no individual gaze labels in the football data, and the final pose landmarks do not measure eye direction.

Why should body points help at all?

Body arrangement can be associated with nearby action, so it is a plausible source of predictive information. The study tests that association. It does not hard-code a pose-to-ball rule or prove a particular joint caused an improvement.

Is an eye point from YOLO an iris measurement?

No. It is an estimated facial landmark location. It does not describe pupil displacement or where the eyeball is pointed.

Which components were pretrained and which were fitted locally?

YOLO and Gaze-LLE use pretrained weights. The full and position-only player selectors and their two offset regressors are fitted locally using the training set.

What is original here?

The project contributes its implemented player-based prediction route and its assessment of selection, features, offsets and blending. It does not claim a new neural architecture or the first use of players to infer a ball.

What is the strongest result to mention?

Raw has 271 close estimates versus centre's 87 across 813 images, but Gaze-LLE has 288 and lower mean error. The evidence is strongest when that useful close-count gain is explained alongside the large-error weakness and the selection diagnostic.

29. Viva questions: data, learning and implementation

Where does ground truth come from?

The centres of the source dataset's labelled physical ball boxes. These are different from judges' estimates in separate missing-ball examples.

Why digitally remove a ball instead of using naturally hidden ones?

The original visible ball has a known coordinate, making scoring straightforward. Natural occlusion is more realistic but needs a defensible hidden location for evaluation. Artificial editing may leave traces.

Does removal itself leak the answer?

The label is used to make the edit and is withheld from ordinary prediction. However, the pixels can retain a trace. The extra-patch check addresses one part of that possibility, not every shortcut.

Why keep whole matches out of fitting?

Frames in one match can share strong information. Match separation reduces direct training/evaluation overlap from related frames.

What does "excluded from training but used in development" mean?

The fitted models in this comparison did not train on those matches. The collection had influenced earlier development, so it is not an untouched external test of all choices.

Why does the expansion not solve that problem?

It adds views from the same withheld matches. It improves coverage of those matches without making them new independent matches or removing earlier exposure.

How was the training player selected?

By the nearest retained player box to the labelled ball, if that distance is at most 0.05 of the pixel diagonal. It is a practical proximity label, not a verified role or gaze label.

How can the selector choose a player when the ball is missing?

It learns from labelled training examples what the assigned player's features tend to look like. On a new edited image it scores each retained player using features only and chooses the highest score.

Why logistic regression?

It provides a straightforward learned score for a fixed numerical feature row and works for the binary proximity label. The model family was inherited from development; the thesis does not establish that it is the best possible selector.

Why standardise and use balanced weights?

Feature scales differ, and there are many more negative player rows than positive ones. Standardisation and class weighting address those two issues using training data.

Does 0.8 from the selector mean an 80% chance of locating the ball correctly?

No calibration of that claim was performed. It is the selector's positive-class output under its training setup, used to rank candidates. It is not calibrated confidence in the final coordinate.

Why just one player?

It gives a fixed reference and feature vector for the offsets. The drawback is that a wrong choice affects the whole prediction and interactions between players are left out.

Why 3,238 images but 1,957 offset examples and 18,540 player rows?

Only 1,957 images meet the near-player training rule. One positive player per such image trains each offset model, while every retained player in those images trains the selector. All 3,238 ball labels contribute to the training mean.

What do the two regressors predict?

Horizontal and vertical ball displacement from the selected box centre in normalised image coordinates. Adding them to that centre gives the raw point.

Why offsets instead of directly predicting x and y?

Offsets give the model a player reference and a local relationship to learn. This is the implemented design; no controlled direct-coordinate comparison establishes its superiority.

Does the program explicitly recognise kicking or heading?

It supplies landmark and box measurements to learned models. Those action labels are not provided, so they should not be described as verified internal reasoning.

Why missing-point flags as well as zero coordinates?

A valid relative coordinate can be zero too. The flag distinguishes a point treated as missing from one treated as present.

What happens when there are no players?

The system uses the training-position mean and keeps the case in the evaluation. There is one original fallback and 16 across the expanded set.

What happens when the ball is far from everyone?

Prediction still runs. Such cases may not resemble the eligible local pairs used for training, which helps explain their larger raw errors. The label is not available to reject them at prediction time.

What does the grass filter really establish?

Only that enough pixels near the box bottom meet a green-colour rule. It roughly favours pitch-side detections and can make mistakes. It does not recognise a player role.

Why keep the shuffle repeatable?

It makes that control reproducible for a particular image. The shuffle changes feature ownership, not the labels or pixels. Its retained seed prefix has no football meaning.

What does models.joblib contain?

The fitted full and position-only selector pipelines, their x/y regressors and the training average. It is reused learned state, rather than a text report of final scores.

30. Viva questions: results and the difficult comparisons

What is raw versus blended prediction?

Raw is centre plus predicted offsets, clipped to the image. Blended is one third raw and two thirds training mean. It combines coordinates, not players.

Why was the blend added, and why is it a weakness?

It was intended to reduce extreme estimates. It also moves good ones away from the ball and restricts the reachable area to about 11.1% of the image. The weight was not validated as optimal.

Why not choose a better weight now?

Using evaluation scores to choose it would make that evaluation part of tuning. A separate match-separated validation set is needed, followed by testing on new matches.

Did the system beat centre guessing?

Raw gives more close estimates. The main blend's mean-error gain is unclear on 61 images and supported as modest on the expanded match-weighted comparison. The output, metric and sample need to be named.

Does Gaze-LLE winning invalidate the work?

It limits the performance claim. The implementation and controls still identify which choices help and where errors enter. A useful evaluation can find that a proposed system is weaker than an established comparison.

Did the body points prove useful?

The fuller route improves on position-only in the expanded comparison. It adds box shape and visibility as well as landmarks and may change player choice, so it does not isolate body pose alone.

Why include both position-only and shuffled controls?

Position-only learns a route without the extra measurements. Shuffling tests the full model when those measurements no longer belong to the correct players. Shuffling can create inconsistent combinations, so the position-only comparison is easier to interpret.

Can more close estimates coexist with a worse mean?

Yes. Very large misses outside the close threshold can outweigh the successful estimates. The close count ignores how large a miss is once it fails the threshold; the mean includes it.

Is error 0.10 a 90% accurate prediction?

No. It is a distance threshold in separately normalised coordinates. It is neither a probability nor a general accuracy percentage.

Why are 32.3% and 271/813 both reported?

32.3% gives every match equal influence. 271/813 counts images equally and is about 33.3%. Matches have different frame counts.

What did answer-assisted selection actually change?

It used the true ball to choose the nearest box, then kept the offset models and point rules. Raw mean falls from 0.2285 to 0.1029 and close count rises from 21 to 34, showing a large selection-related effect.

Is 0.1029 the accuracy of the real deployed method?

No. It needs the answer to choose the player. The actual raw result on the original sample is 0.2285.

How much of the assisted result is simply proximity?

A large part. The assisted player's centre alone gives 0.1150 mean and 34 close; offsets give 0.1029 and the same close count. The offsets add a smaller improvement in mean error.

Do offsets beat using the actual selected player's centre?

On the original sample they improve mean error, 0.2285 versus 0.2417, but reduce close count from 23 to 21. On the expanded sample both reported measures improve.

Why not report only the 37 eligible images?

Those cases are identified using the known ball location. Keeping every evaluation image exposes the training-coverage weakness rather than removing hard cases with knowledge of the answer.

Is a zero-crossing interval evidence that two methods are the same?

No. It means the comparison does not clearly determine the direction of the difference under the resampling procedure. Demonstrating equivalence needs a different explicit design.

Why bootstrap matches rather than individual frames?

Frames in a match are related. Whole-match resampling keeps that grouping and paired draws compare both methods on the same sampled information.

Does the bootstrap solve development bias?

No. It describes variation over the observed matches while treating the chosen procedure as fixed. It does not undo earlier exposure or test different development choices.

Does the extra-patch result prove Gaze-LLE understood football?

No. It shows no attraction of the strongest point to that added edit. It neither identifies all features used nor rules out help from the original removal trace.

Why is the head-marker comparison restricted to 49?

Only 49 historical person-marked outputs were available. Every method uses those same images in that table; the remaining original cases stay in the main unmarked comparison.

Why not compare 39/61 heatmap coverage with 24/61 close points?

They measure different things. The former allows the top-scoring 10% of image area; the latter checks a single coordinate under distance 0.10.

Did supplying heads generally make Gaze-LLE worse?

The observed mean for the historical person-map average is higher on this matched sample. That does not establish a general rule about markers or every way of combining person maps.

What do the current code/thesis differences mean for reproducibility?

The current shortened detector omits the thesis tiles, and head preparation also changed. These can alter inputs and outputs. Existing model compatibility does not demonstrate exact recovery of the published tables.

31. Viva questions: contribution, future work and honest boundaries

Why not train directly from image pixels?

That would be a closer learned baseline on the same task, and the thesis proposes it. It was not implemented here, so the study cannot conclude that player features beat every image-based approach.

Why not use video tracking?

Video adds motion and temporal context. This task deliberately predicts from a single edited image. Tracking is a related alternative with extra information rather than a directly equivalent evaluation.

What should be improved first?

Player-selection supervision and training coverage. The assisted result shows a larger effect from player choice than the smaller offset gain. The fixed blend then needs validation rather than automatic retention.

Would using all players solve the problem?

It could reduce dependence on one choice, but would need a method for weighting reliable cues and handling people looking elsewhere. The claim needs evaluation; more players do not automatically mean more useful evidence.

Would players looking in the same direction identify the ball?

Only as a clue. A common target can lie in different directions from different people, and similar projected directions can point to different targets. Agreement on a location with uncertainty is the more useful goal.

What about players looking at one another?

That may reveal a competing interaction rather than ball attention. A future system could test whether recognising it helps select useful cues, without assuming every face is ball-directed.

Could synthetic faces solve the training problem?

They can supply controlled head/eye labels and diagnostic cases, but realism and transfer to real faces need testing. The submitted thesis does not show that a synthetic training scheme improved this final system.

What would a stronger final evaluation look like?

Development and validation on separate matches, a locked procedure tested on genuinely new matches, natural occlusion with reliable labels and a direct image-to-ball baseline. A user study would assess practical benefit separately.

What if the panel asks something the study did not test?

State the part that is known and identify what evidence is missing. A plausible explanation is a hypothesis until it has been checked. Do not turn an implementation choice into an invented experiment.

What is the overall conclusion in one answer?

Player information gives useful close estimates but the current system is uneven. Its reference-player choice and local training coverage contribute large errors, while the fixed blend loses good estimates. Fuller features and learned offsets help modestly in the expanded comparison, and Gaze-LLE has lower mean error.

32. A small glossary for reading the code and thesis

Term	Meaning here
Annotation / label / ground truth	Recorded physical ball-box centre, or a derived training target
Frame / image	One still input; the system does not use neighbouring frames
Bounding box	Detected or annotated rectangle around a person or ball
Landmark / keypoint	Estimated location of a body or facial point
Feature / representation	A numerical description supplied to the learning models
Visibility flag	Whether a point passes the confidence rule; not a manual visibility label
Selector	Model that scores candidates and chooses one player
Regressor	Model that predicts a numerical value, here an x or y offset
Offset	Ball coordinate minus reference-player centre coordinate
Raw	Player centre plus predicted offsets, after clipping
Blend / full_fusion in older files	Raw coordinate pulled towards the training mean
Prior in older files	The training-position mean, not a learned gaze direction
Fallback	Training mean used when no player remains
Baseline	A comparison method, such as centre or Gaze-LLE
Ablation / control	Change that helps examine one part of a method
Eligibility	Known ball is near enough to a retained box under the training rule
Answer-assisted / oracle wording in old records	Nearest-player choice using the true ball; a diagnostic
Decoy / extra patch	Second inpainted area used in the editing check
Scene / no_head / null wording in old records	Gaze-LLE input without a supplied person marker
Head marker / head prompt	Box telling Gaze-LLE which person's gaze to condition on
Heatmap peak	Highest-scoring cell converted to a coordinate
Joint attention	Several people attending to a shared target
Head pose	Orientation of the head; can differ from eye gaze

Term	Meaning here
Leakage	Information reaching a comparison in a way that makes its conclusion too favourable
Validation	Separate development data for choosing settings
Evaluation / test	Data used to assess a specified procedure
Generalisation	Performance beyond the examples and conditions used in development
Match-weighted	Summarise within matches, then give each match equal influence
Paired	Compare methods on the same images or resampled matches
Bootstrap	Repeatedly resample observed units to describe variation
Pointwise interval	Uncertainty statement for an individual comparison
Percentage points / pp	Difference between two percentage rates
Replay	Recalculate a saved procedure to check consistency
Cache / reused output	Saved work loaded instead of performing that step again

File extensions: .py is source code; .md is documentation; .json stores readable records; .csv stores annotation/table rows; .joblib stores local fitted estimators; .pt/.pth store neural weights; .npz stores arrays such as heatmaps; .png stores prepared images or figures. The downloaded source and weights are assets, not new models invented by the project.

33. How to learn this without memorising a script

First explain the problem without model names. Say what has been removed, what remains, what the system predicts and where the scoring label comes from. Then draw the six-stage route on paper.

Next separate the three learning questions: **which player, what displacement, how much central pull**. Use the illustrative coordinates to calculate a raw and blended point. Explain why the body points are predictive measurements rather than an explicit gaze calculation.

Then practise the result trade-off. Give one score for raw, one for the main blend and one for Gaze-LLE, naming the sample and metric. Explain how a method can have more close estimates and still lose on mean error.

Follow one image through main.py using the file map. Find its record, retained players, feature rows, selected player, raw point and scored error. Reading saved records is different from running inference, and scoring old outputs is different from refitting models.

Finally use the questions as a self-test. Give an answer aloud before reading the supplied answer. If the answer relies on a term that is still vague, return to the explanation where it first appears. The goal is to understand the connection, not reproduce this guide's exact wording.

Check that these can be explained without looking

- Why 813 includes the original 61 and still represents 61 matches
- Why the training ball label is needed but cannot choose a player during real prediction
- Why 18,540 player rows and 1,957 offset examples are different counts
- What a body point's coordinate is measured relative to
- How a score selects a player and how two offsets become a raw point
- Why one-third blending limits predictions to 11.1% of image area
- Why 271/813 and 32.3% are different summaries
- Why answer-assisted 0.1029 needs the nearby-centre comparison of 0.1150
- What the extra-patch check leaves unresolved
- Which implementation differences prevent an automatic exact-reproduction claim for current GitHub code

If those explanations are clear, the central research and implementation story is in place. The appendix values and seeds can be looked up rather than memorised.

34. The short revision page

Task: infer the location of a digitally removed physical ball in one football image. Labels come from SoccerNet-v3D ball-box centres.

Method: pretrained YOLO pose -> numerical player features -> learned player choice -> learned x/y offsets -> clipped raw point -> optional fixed blend towards the training-ball mean. Gaze-LLE is a separate scene-based comparison.

Training: 3,238 images, 318 matches. Near-player rule leaves 1,957 offset examples and 18,540 selector rows. Evaluation: original 61 images from 61 other matches, expanded to 813 images from those same matches. Earlier development exposure remains.

Original result: raw 21/61 close and mean 0.2285; main blend 6/61 and 0.2131; centre 5/61 and 0.2302; Gaze-LLE 24/61 and 0.1625. The blend's mean-error gain over fixed baselines is unclear on this sample.

Expanded result: raw 271/813 close, match mean 0.2294; blend 125/813, 0.2125; centre 87/813, 0.2267; Gaze-LLE 288/813, 0.1909. The blend's modest gains over centre and position-only are supported. More close predictions do not remove raw's larger-error problem.

Why failures occur: wrong reference player, local training pairs that do not cover distant-ball images, and an untuned blend that pulls good estimates centrally.

Selection evidence: actual raw 0.2285 and 21 close -> answer-assisted raw 0.1029 and 34 close. However, the assisted centre already gives 0.1150 and the same 34. Selection carries much of the gain; offsets add a smaller one.

Own contribution: the implemented player-based system and the comparisons that identify what its stages contribute. Established detectors and learning algorithms were reused and the local models were fitted; no new gaze architecture is claimed.

Next step: better player labels and training coverage, separate match validation for blending, a direct image baseline and an external test. Multi-player agreement is a reasonable additional proposal with existing related work and its own validation needs.

Code boundary: current GitHub detection is a simplified one-pass route; the thesis used tiles as well. The current repository does not automatically establish exact reproduction of the published tables.

35. Sources and coverage

Primary document: **Thesis.pdf**, uploaded at https://github.com/Juice234/Quorum-Gaze/blob/2c57c02fda176855126c4f883fb527cbc2151481/Thesis.pdf. All 45 PDF pages, including the bibliography and Appendix A, were read. Numeric thesis results in this guide refer to that document.

Code source: **the complete current repository tree** at https://github.com/Juice234/Quorum-Gaze/tree/2c57c02fda176855126c4f883fb527cbc2151481. All 34 Python, Markdown, TOML and requirements files, plus three Git configuration files, were read. This includes active and deprecated source. The binary recording and saved comparison image were inventoried, not used as evidence for a new numerical claim.

Comparison source: the supplied **Football_Project_Learning_Guide.pdf**, 31 pages, was read in full before this revision. Its explanations were checked against the uploaded thesis and repository rather than assumed current.

Thesis coverage	Where it is taught in this guide
Chapter 1: task, motivation, question, scope	Sections 1-2 and 27
Chapter 2: data, labels, methods, metrics, comparisons	Sections 3-4 and 7-13
Chapter 3: related approaches and evaluation practice	Section 21, with relevant explanations throughout
Chapter 4: earlier gaze route and final implementation	Sections 2 and 4-11; code map in 22-26
Chapter 5: all main tables, diagnostic checks and discussion	Sections 14-20 and 27
Chapter 6: conclusions, contribution and future work	Sections 27-31 and 34
Appendix A: settings, paired comparisons, data use	Sections 12-20 and 26; exact intervals remain in the source appendix
Current code, setup, file roles and output interpretation	Sections 22-26 and glossary
Practice explaining and defending the work	Sections 28-34

This guide explains the study and the questions most directly raised by its evidence. It cannot predict every viva question or replace understanding the submitted document. When an exact interval or unremembered implementation detail is requested, the source appendix and the recorded code remain the reference.